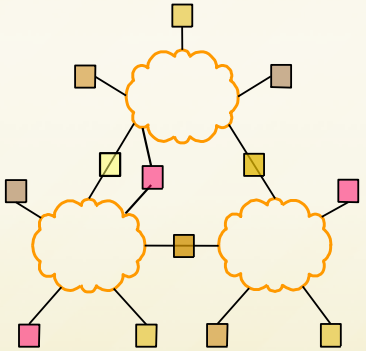
# 15-440 Distributed Systems Fall 2016



L-23 Security

# What do we need for a secure communication channel?

- Authentication (Who am I talking to?)
- Confidentiality (Is my data hidden?)
- Integrity (Has my data been modified?)
- Availability (Can I reach the destination?)

# Example: Web access

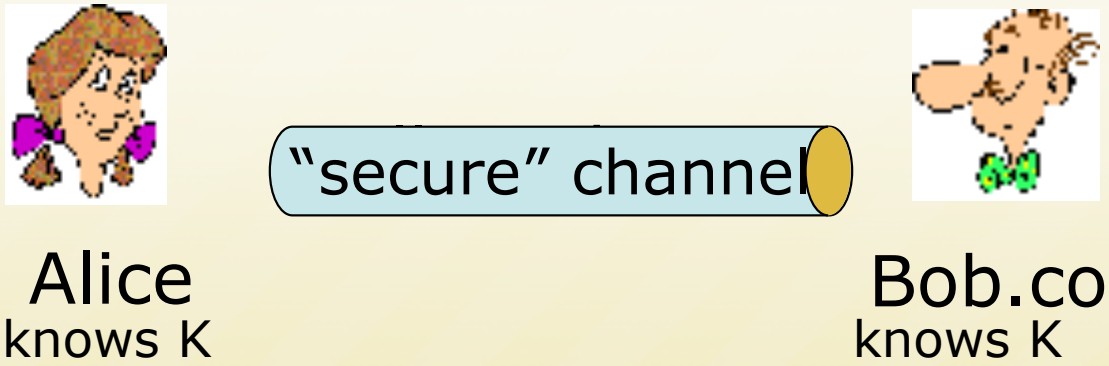
- Alice wants to connect to her bank to transfer some money...
- Alice wants to know ...
  - that she's really connected to her bank. Authentication
  - That nobody can observe her financial data Confidentiality
  - That nobody can modify her request Integrity
  - That nobody can steal her money! (A mix)
- The bank wants to know ...
  - That Alice is really Alice (or is authorized to act for Alice)
  - The same privacy things that Alice wants so they don't get sued or fined by the government.

# How do we create secure channels?

- What tools do we have at hand?
- Hashing
  - e.g., SHA-1
- Secret-key cryptography, aka symmetric key.
  - e.g., AES
- Public-key cryptography
  - e.g., RSA

# Secret Key Cryptography

- Given a key k and a message m
  - Two functions: Encryption (E), decryption (D)
  - ciphertext c = E(k, m)
  - plaintext m = D(k, c)
  - Both use the same key k.




But... how does that help with authentication?

They both have to know a pre-shared key K before they start!

# Symmetric Key: Confidentiality

- One-time Pad (OTP) is secure but usually impractical
  - Key is as long at the message
  - Keys cannot be reused (why?)

In practice, two types of ciphers are used that require only constant key length:

**Stream Ciphers:**

Ex: RC4, A5

**Block Ciphers:**

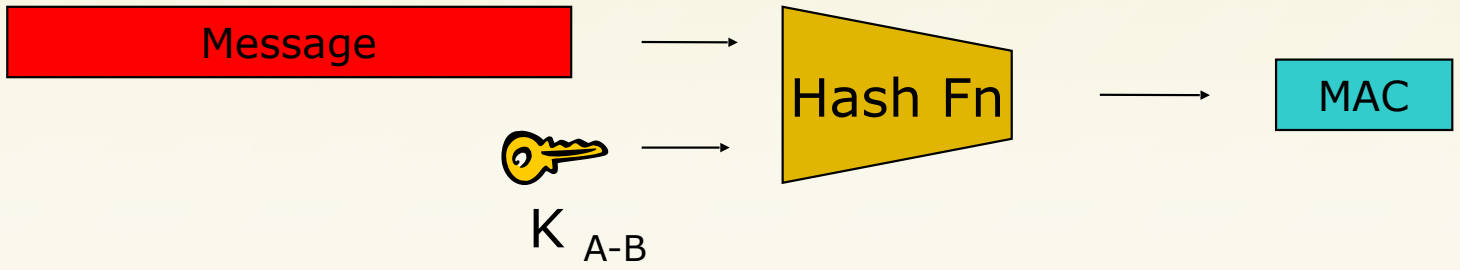
Ex: DES, AES, Blowfish

# Symmetric Key: Integrity

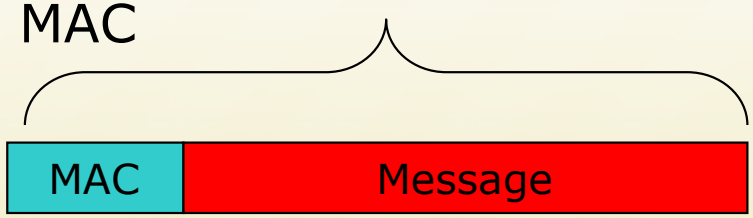
- Hash Message Authentication Code (HMAC)

Step #1:

Alice creates MAC




Step #2 Alice Transmits Message & MAC



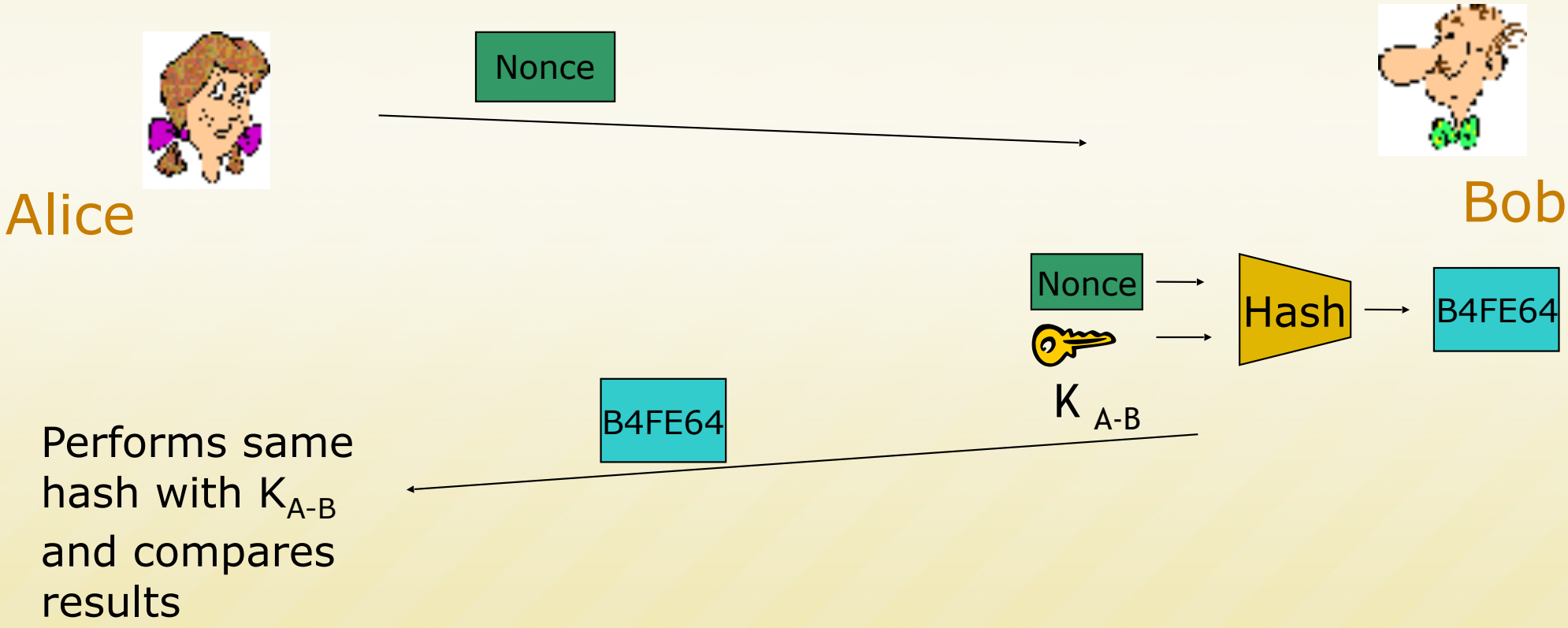

Step #3

Bob computes MAC with message and $K_{A-B}$ to verify.

Why is this secure from a message integrity perspective? How do properties of a hash function help us?

# Symmetric Key: Authentication

- A "Nonce"
  - A random bitstring used only once. Alice sends nonce to Bob as a "challenge". Bob Replies with "fresh" MAC result.



# Symmetric Key Crypto Review

- Confidentiality:  Stream & Block Ciphers
- Integrity:  HMAC
- Authentication: HMAC and Nonce

**Are we done?  Not Really:**

**1) Number of keys scales as $O(n^2)$**

**2) How to securely share keys in the first place?**

# Asymmetric Key Crypto:

- Instead of shared keys, each person has a "key pair"



 $K_B$ Bob's public key

 $K_B^{-1}$ Bob's private key

- The keys are inverses, so: $K_B^{-1}(K_B(m)) = m$

# Asymmetric/Public Key Crypto:

- Given a key *k* and a message *m*
  - Two functions: Encryption (E), decryption (D)
  - ciphertext $c = E(K_B, m)$
  - plaintext $m = D(K_B^{-1}, c)$
  - Encryption and decryption use *different* keys!



"secure" channel

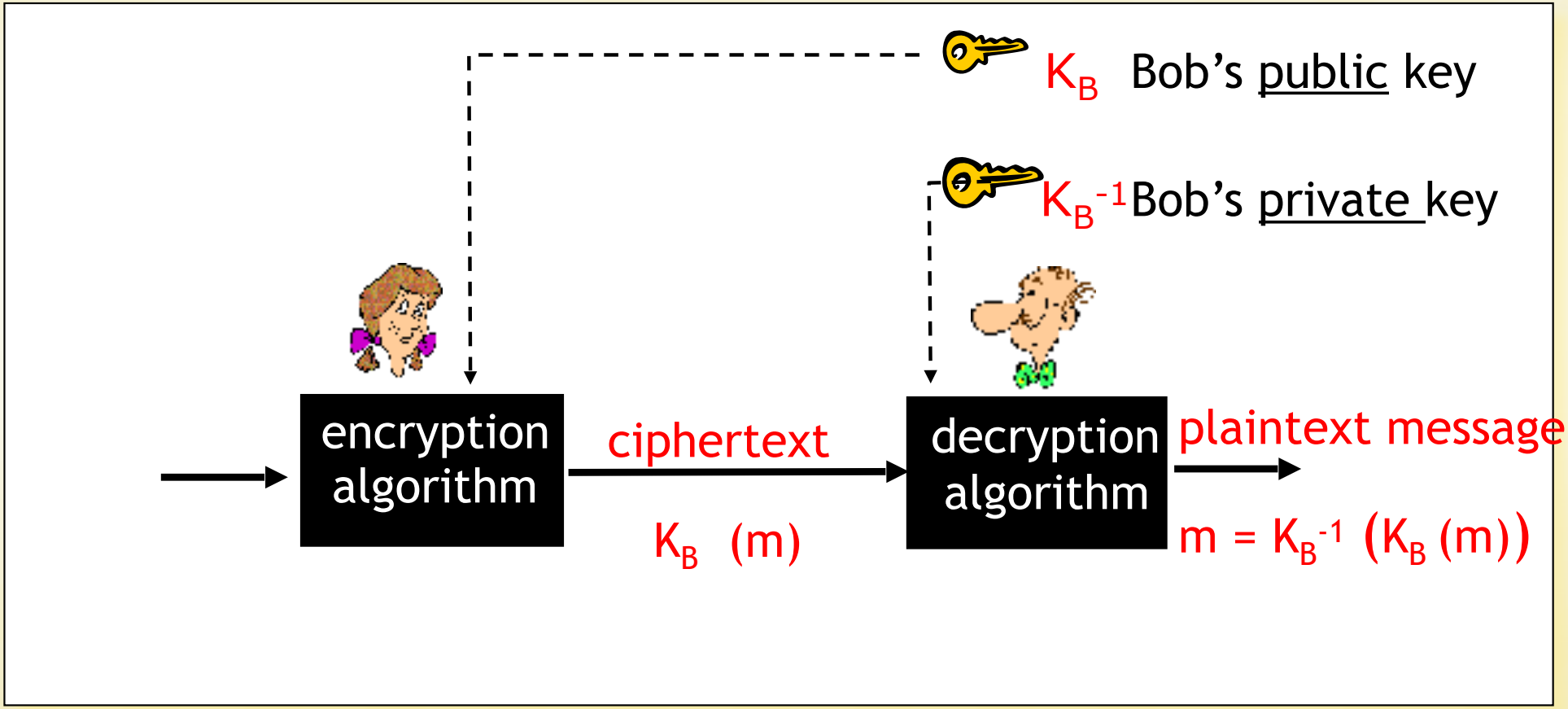


Alice
Knows $K_B$

Bob.com
Knows $K_B$, $K_B^{-1}$

But how does Alice know that $K_B$ means "Bob"?

# Asymmetric Key: Confidentiality



# Asymmetric Key: Sign & Verify

- If we are given a message M, and a value S such that $K_B(S) = M$, what can we conclude?
  - The message must be from Bob, because it must be the case that $S = K_B^{-1}(M)$, and only Bob has $K_B^{-1}$ !
  - This gives us two primitives:
    - Sign (M) = $K_B^{-1}(M)$ = Signature S
    - Verify (S, M) = test( $K_B(S)$ == M )

# Asymmetric Key: Integrity & Authentication

- We can use Sign() and Verify() in a similar manner as our HMAC in symmetric schemes.

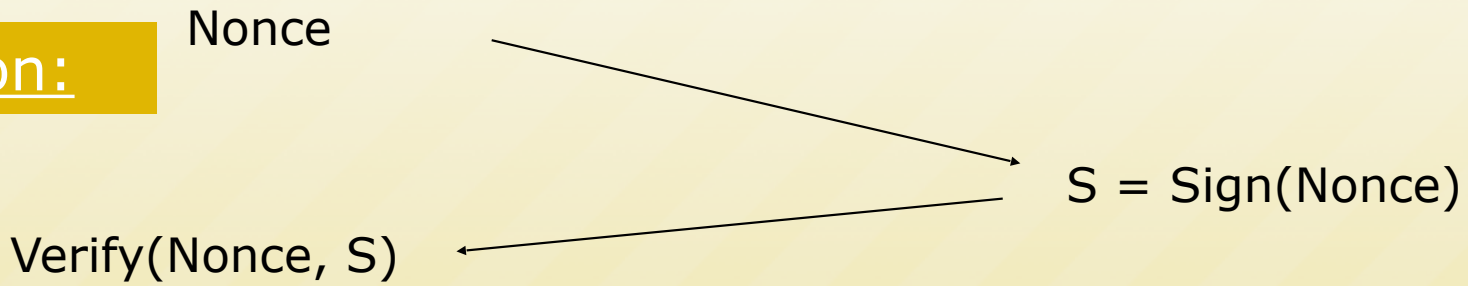
Integrity:




Receiver must only check Verify(M, S)

---

Authentication:



# Asymmetric Key Review:

- Confidentiality: Encrypt with Public Key of Receiver
- Integrity: Sign message with private key of the sender
- Authentication: Entity being authenticated signs a nonce with private key, signature is then verified with the public key

But, these operations are computationally expensive*

# The Great Divide

| | Symmetric Crypto: (Private key) Example: AES | Asymmetric Crypto: (Public key) Example: RSA |
|---|---|---|
| Requires a pre-shared secret between communicating parties? | Yes | No |
| Overall speed of cryptographic operations | Fast | Slow |

# One last “little detail”…

How do I get these keys in the first place?? Remember:

- Symmetric key primitives assumed Alice and Bob had already shared a key.
- Asymmetric key primitives assumed Alice knew Bob’s public key.

This may work with friends, but when was the last time you saw Amazon.com walking down the street?

# Key Distribution Center (KDC)

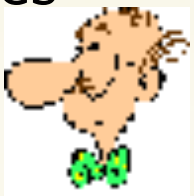
*Q:* How does KDC allow Bob, Alice to determine shared symmetric secret key to communicate with each other?



$K_{A\text{-}KDC}(A,B)$



KDC generates R1



$K_{A\text{-}KDC}(R1, K_{B\text{-}KDC}(A,R1))$

Alice knows R1

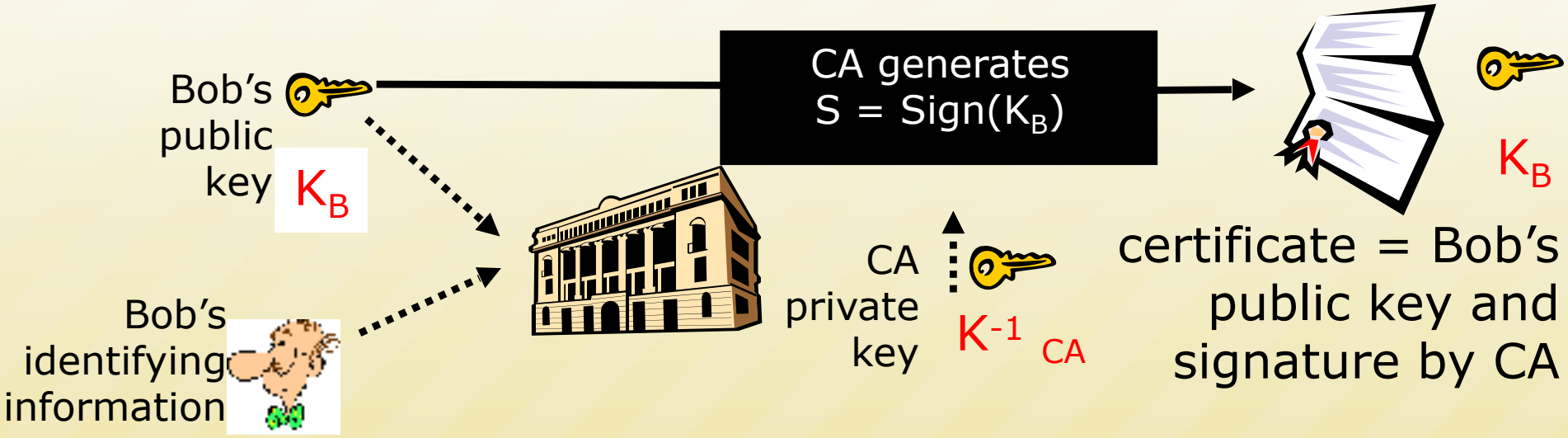
Bob knows to use R1 to communicate with Alice

$K_{B\text{-}KDC}(A,R1)$

Alice and Bob communicate: using R1 as *session key* for shared symmetric encryption

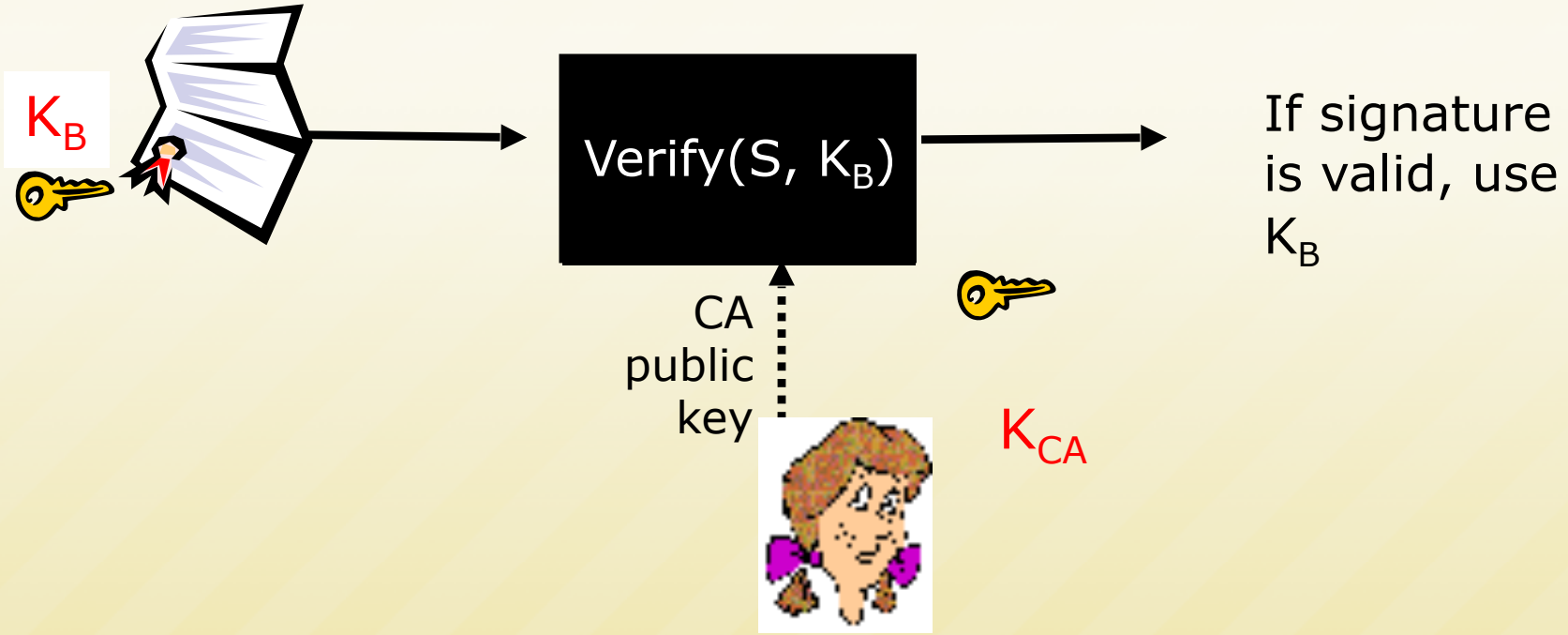
# Certification Authorities

- Certification authority (CA): binds public key to particular entity, E.
- An entity E registers its public key with CA.
  - E provides "proof of identity" to CA.
  - CA creates certificate binding E to its public key.
  - Certificate contains E's public key AND the CA's signature of E's public key.



# Certification Authorities

- When Alice wants Bob's public key:
  - Gets Bob's certificate (Bob or elsewhere).
  - Use CA's public key to verify the signature within Bob's certificate, then accepts public key



# Transport Layer Security (TLS) aka Secure Socket Layer (SSL)

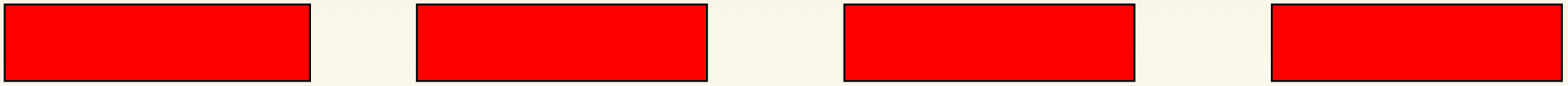
- Used for protocols like HTTPS
- Special TLS socket layer between application and TCP (small changes to application).
- Handles confidentiality, integrity, and authentication.
- Uses “hybrid” cryptography.

# How TLS Handles Data

1) Data arrives as a stream from the application via the TLS Socket



2) The data is segmented by TLS into chunks



3) A session key is used to encrypt and MAC each chunk to form a TLS "record", which includes a short header and data that is encrypted, as well as a MAC.



4) Records form a byte stream that is fed to a TCP socket for transmission.



# Analysis

- Public key lets us take the trusted third party offline:
  - If it's down, we can still talk!
  - But we trade-off ability for fast revocation
    - If server's key is compromised, we can't revoke it immediately...
    - Usual trick:
      - Certificate expires in, e.g., a year.
      - Have an on-line revocation authority that distributes a revocation list. Kinda clunky but mostly works, iff revocation is rare. Clients fetch list periodically.
- Better scaling: CA must only sign once... no matter how many connections the server handles.
- If CA is compromised, attacker can trick clients into thinking they're the real server.

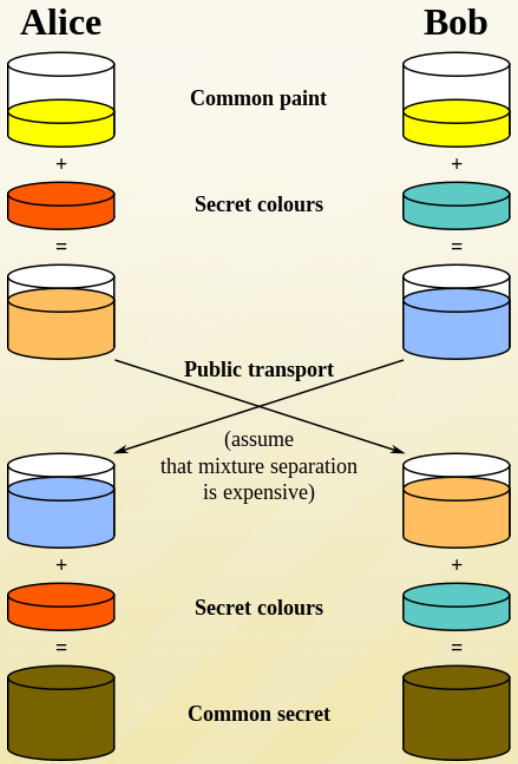
# Forward secrecy

- In KDC design, if key $K_{server\text{-}KDC}$ is compromised a year later,
  - from the traffic log, attacker can extract session key (encrypted with auth server keys).
  - attacker can decode all traffic retroactively.

- In SSL, if CA key is compromised a year later,
  - Only new traffic can be compromised. Cool…
- But in SSL, if server's key is compromised...
  - Old logged traffic can still be compromised...

# Diffie-Hellman Key Exchange

- Different model of the world: How to generate keys between two people, securely, no trusted party, even if someone is listening in.




Illustrative Example
image from wikipedia
https://en.wikipedia.org/wiki/Diffie%E2%80%93Hellman_key_exchange

# Diffie-Hellman Key Exchange

- Different model of the world: How to generate keys between two people, securely, no trusted party, even if someone is listening in.

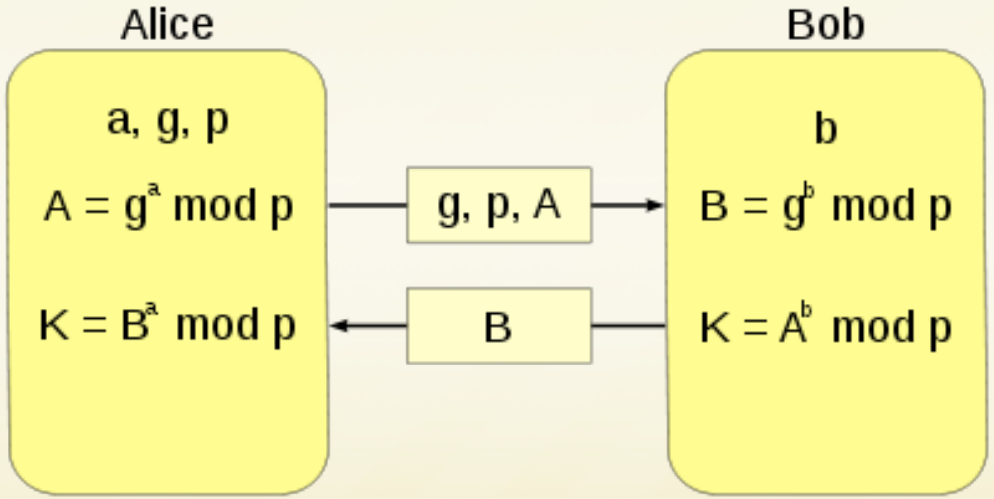



image from wikipedia

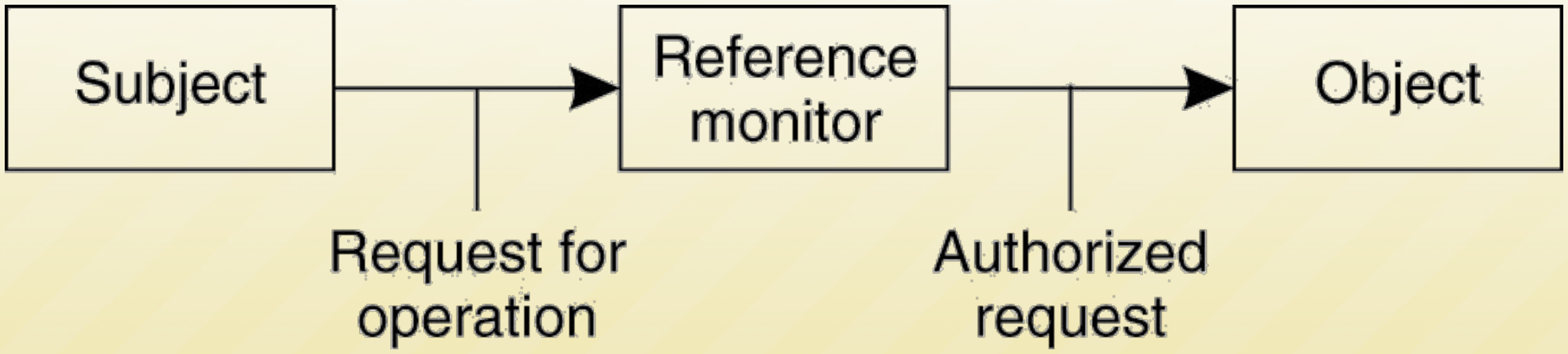
- This is cool. But: Vulnerable to man-in-the-middle attack. Attacker pair-wise negotiates keys with each of A and B and decrypts traffic in the middle. No authentication...

# Authentication?

- But we already have protocols that give us authentication!
  - They just happen to be vulnerable to disclosure if long-lasting keys are compromised later...
- Hybrid solution:
  - Use diffie-hellman key exchange with the protocols we've discussed so far.
  - Auth protocols prevent M-it-M attack if keys aren't yet compromised.
  - D-H means that an attacker can't recover the real session key from a traffic log, even if they can decrypt that log.
  - Client and server discard the D-H parameters and session key after use, so can't be recovered later.
- This is called "perfect forward secrecy". Nice property.

# Access Control

- Once secure communication between a client and server has been established, we now have to worry about access control – when the client issues a request, how do we know that the client has **authorization**?

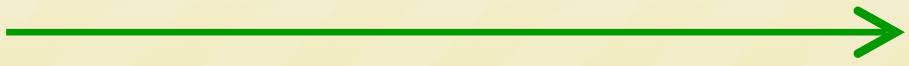


# Two ways to cut a table (ACM)

- Order by columns (ACL) or rows (Capability Lists)?

| | File*1* | File*2* | File*3* |
|---|---|---|---|
| *Ann* | *rx* | *r* | rwx |
| *Bob* | *rwx* | r | -- |
| *Charlie* | *rx* | rw | w |



# Access Control Lists

- An ACL stores (non-empty elements of) each column with its object
- Columns of access control matrix

| | File*1* | File*2* | File*3* |
|---|---|---|---|
| *Andy* | *rx* | *r* | rwx |
| *Betty* | *rwx* | r | -- |
| *Charlie* | *rx* | rw | w |

- ACLs:
- file1: { (Andy, rx) (Betty, rwx) (Charlie, rx) }
- file2: { (Andy, r) (Betty, r) (Charlie, rw) }
- file3: { (Andy, rw) (Charlie, w) }

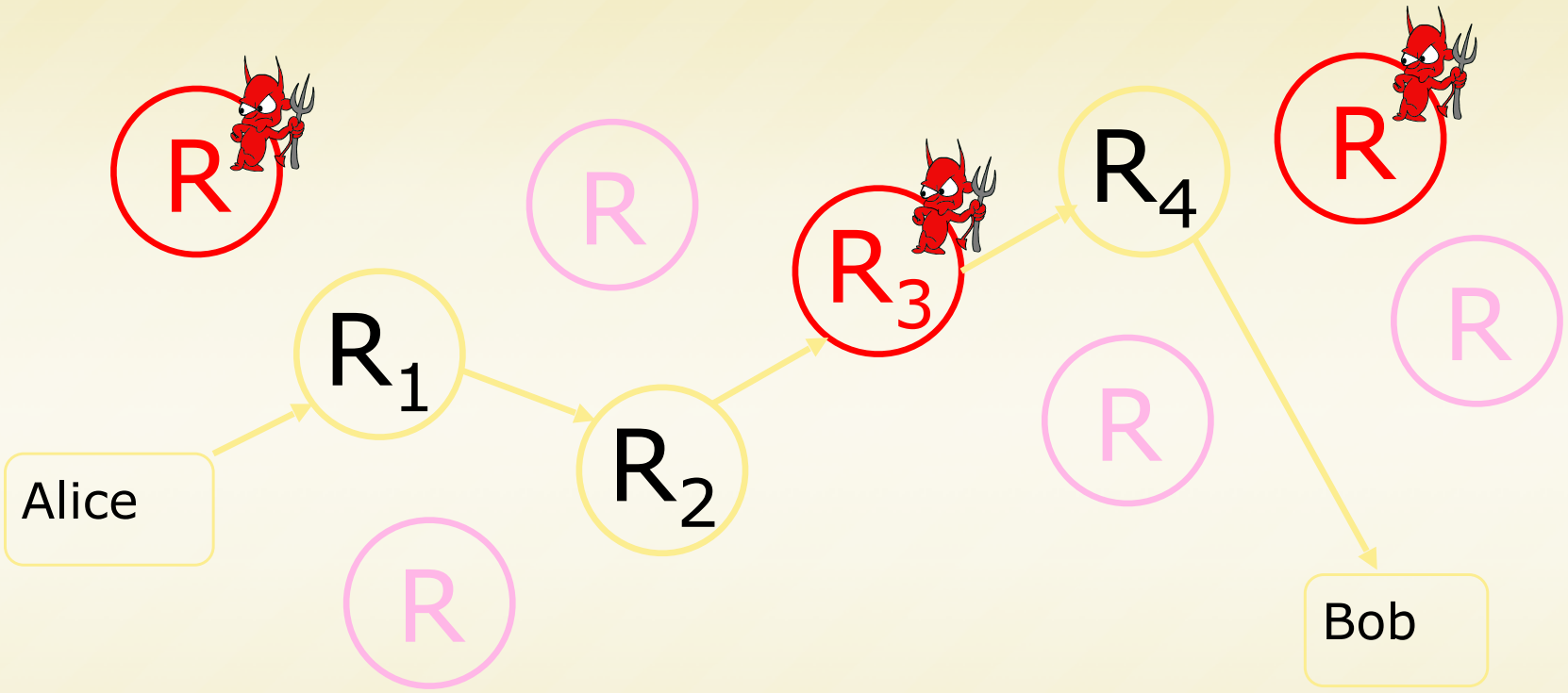
# Capability Lists

- Rows of access control matrix

| | File*1* | File*2* | File*3* |
|---|---|---|---|
| *Andy* | *rx* | *r* | rwx |
| *Betty* | *rwx* | r | -- |
| *Charlie* | *rx* | rw | w |

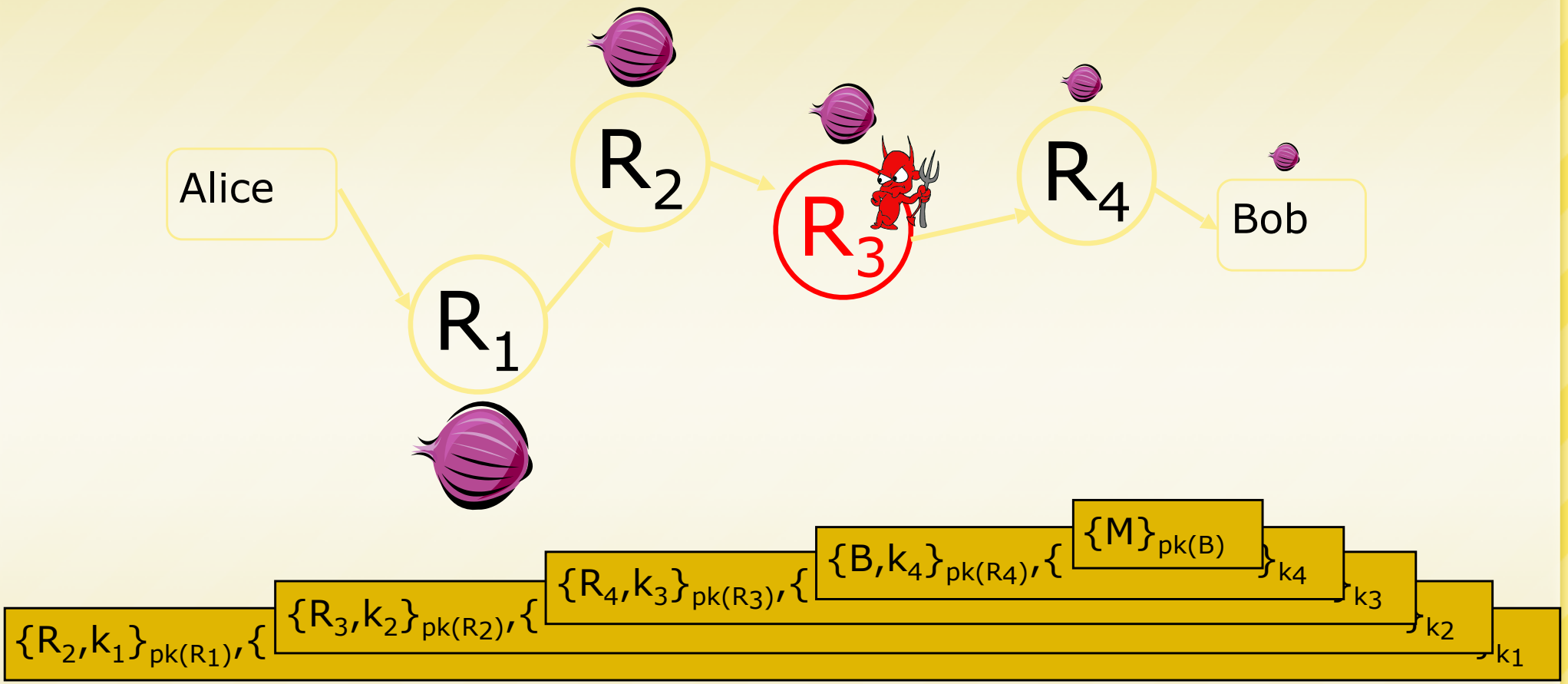
- C-Lists:
  - Andy: { (file1, rx) (file2, r) (file3, rw) }
  - Betty: { (file1, rwx) (file2, r) }
  - Charlie: { (file1, rx) (file2, rw) (file3, w) }

# Onion Routing




- Sender chooses a random sequence of routers
  - Some routers are honest, some controlled by attacker
  - Sender controls the length of the path

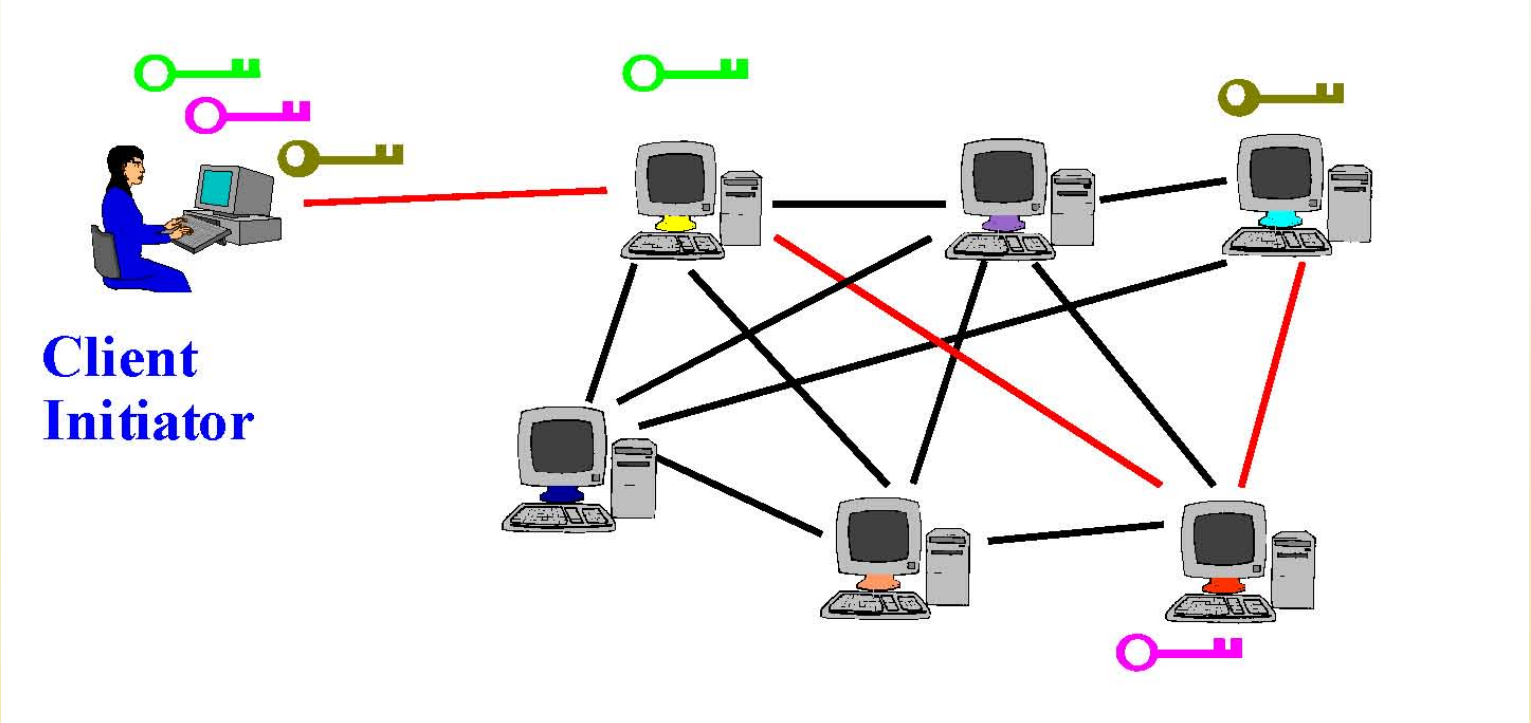
# Route Establishment




Routing info for each link encrypted with router's public key
Each router learns only the identity of the next router

# Tor Circuit Setup (3)

- Client proxy extends the circuit by establishing a symmetric session key with Onion Router #3
  - Tunnel through Onion Routers #1 and #2



# Key Bits: Today's Lecture

- Effective secure channels
  - Key Distribution Centers and Certificate Authorities
  - Diffie-Hellman for key establishment in the "open"
- Access control
  - Way to store what "subjects" can do to "objects"
  - Access Control Matrix: ACLs and Capability lists
- Privacy and Tor
  - Used for anonymity on the internet (Onion Routes)
  - Uses ideas from encryption, networking, P2P